IT305 – COMPUTER NETWORKS LAB

# FINAL PROJECT REPORT

Fault-Tolerant Topic-Based
File Distribution Service

---

Project groups
Group 078 + Group 074

Implementation platform
C, POSIX TCP sockets, POSIX threads, Linux

**Report basis.** This report describes the implementation in the submitted project archive. The supplied sample CSV is identified as sample data; it is not presented as a newly reproduced benchmark. The separate design document was used only as a guide for report structure and clarity.

## Group Members and Individual Contributions

The workstreams below reflect the contribution mapping previously provided for this project. The team should confirm that each description matches the work actually completed before submission.

| Member name | Roll number | Contribution / workstream |
|---|---|---|
| Shyam Ramani | 202401175 | Protocol design; Part I server architecture and sequential/concurrent serving. |
| Sherashiya Ramit | 202401198 | Client transfer workflow, framed-message handling, and input/path validation. |
| Om Patel | 202401150 | Part II Case 1 fault injection, reconnect-and-restart behaviour, and transfer metrics. |
| Dev Fultariya | 202401254 | Part II Case 2 session state, checkpoint validation, and byte-offset resume. |
| Yagnik Arvadiya | 202401252 | Case 2 Enhanced parallel TCP streams, chunk scheduling, and concurrent file reassembly. |
| Khush Vadhadiya | 202401267 | Benchmark scripts, CSV analysis, performance graphs, and report integration. |

**Contribution note.** The archive itself contains placeholder group/member labels in its Markdown template. Names and roll numbers above are retained from the project context and should be checked by the team.

**Project at a glance.** The implementation is organised as four progressively more capable TCP file-transfer variants: a sequential/concurrent baseline, restart-after-failure, session-based checkpoint resume, and a parallel-worker extension. The report describes their protocol, concurrency model, recovery behaviour, verification status, and the limits of the currently supplied performance data.

# Contents

**Group Members and Individual Contributions** **1**

**1 Executive Summary** **3**

**2 Requirements and Assumptions** **3**

**3 System Architecture** **4**

**4 Application Protocol** **4**

**5 Part I: Sequential and Concurrent Serving** **6**

**6 Part II Case 1: Fault Injection and Restart** **7**

**7 Part II Case 2: Session and Checkpoint Resume** **7**

**8 Case 2 Enhanced: Parallel Worker Connections** **8**

**9 Input Validation and Concurrency** **8**

**10 Build and Run Guide** **8**

**11 Performance Metrics and Sample Results** **9**

**12 Verification and Limitations** **10**

**13 Design Choices** **12**

**14 Conclusion** **13**

**A Protocol Constants and Bounds** **13**

**B Archive Inventory** **13**

# 1. Executive Summary

This project implements a topic-based file distribution service in C using TCP over IPv4. Each subdirectory under a server root represents a topic. A client requests a topic, and the server sends the regular files stored in that directory.

The project has four variants. Part I compares a single-threaded server with a multi-threaded server. Part II Case 1 adds random connection failures; after a failure, the client starts the topic again. Case 2 adds session information and checkpoints so a client can resume from a saved file and byte offset. Case 2 Enhanced adds parallel TCP worker connections and explicit-offset file writes.

The source archive includes code, Makefiles, a benchmark script, sample results, and plotting scripts. All four variants built in the report-generation environment. A small local smoke test with failure probability zero completed for all four variants. The sample CSV is discussed separately because it was not independently reproduced and the full test dataset is not included in the ZIP.

# 2. Requirements and Assumptions

## 2.1 Main requirements

- Map each topic name to a directory below the configured topic root.
- Transfer the files in a requested topic to the client's output directory.
- Support sequential and concurrent clients in Part I.
- Simulate connection failures in Part II.
- Compare restarting from the beginning with checkpoint-based resume.
- Support multiple worker connections in the enhanced variant.
- Record transfer time, bytes, retries, throughput, and redundant data for evaluation.

## 2.2 Runtime assumptions

The code targets Linux/POSIX systems with a C compiler, pthreads, TCP/IPv4 sockets, and standard filesystem calls. Topic roots, ports, IP addresses, topic names, and output directories are supplied at launch. The client writes received files beneath the selected output directory and topic folder.

## 2.3 Repository structure

| Path | Purpose |
|---|---|
| `Part1/` | Baseline client/server; supports single and multi modes. |
| `Part2_Case1/` | Fault injection without retained transfer progress. |

| | |
|---|---|
| `Part2_Case2/` | Session-aware transfer with checkpoint and resume fields. |
| `Part2_Case2_Enhanced/` | Parallel worker-channel variant with shared task/session state. |
| `Results/` | Benchmark runner, plotting code, sample CSV, and sample graphs. |
| `README.md` | Build and launch instructions. |

# 3. System Architecture

## 3.1 Topic discovery

The server scans the configured root directory and identifies topic subdirectories. It then lists regular files for the requested topic and collects file metadata. The implementation uses directory and filesystem APIs such as `opendir()`, `readdir()`, and `stat()`.

Topic names are validated before they are used to build filesystem paths. This reduces the risk of accepting malformed names or path traversal strings. File names are taken from the server's directory listing rather than trusted as arbitrary client-supplied paths.

## 3.2 End-to-end workflow

1. The server starts with a port and topic-root directory. Part II servers also accept a failure probability.
2. The client connects and sends a framed `MSG_GET` request for a topic.
3. The server validates the request and returns topic/file metadata.
4. The client sends `MSG_START` when it is ready to receive data.
5. The server sends file headers and data chunks. The client writes each chunk and acknowledges progress.
6. After the files are sent, the server sends `MSG_CLOSE` and the client reports the result.

# 4. Application Protocol

## 4.1 Common message header

Each variant defines a packed 28-byte message header in its `protocol.h`. The header carries a protocol marker, message type, flags, session ID, transaction ID, and payload length. The structure is packed so the compiler does not insert padding between fields.

| Field | Size | Purpose |
|---|---|---|
| `magic` | 4 bytes | Protocol identifier, `0x49543330`. |
| `msg_type` | 2 bytes | Identifies the message. |
| `status_flags` | 2 bytes | Reserved status/flag field. |
| `session_id` | 8 bytes | Associates messages with a transfer session. |
| `transaction_id` | 8 bytes | Correlates a message or data chunk. |
| `payload_len` | 4 bytes | Length of the payload after the header. |

### 4.2 Main message types

| Message | Direction | Purpose |
|---|---|---|
| `MSG_GET` | Client to server | Requests a topic or provides resume information. |
| `MSG_OK` | Server to client | Confirms the request and returns topic/session metadata. |
| `MSG_ERR` | Server to client | Reports invalid requests or transfer errors. |
| `MSG_START` | Client to server | Signals that the client is ready for data. |
| `MSG_FILE_HDR` | Server to client | Describes the file and the starting offset. |
| `MSG_DATA` | Server to client | Carries a data chunk and its offset information. |
| `MSG_ACK` | Client to server | Acknowledges received data and progress. |
| `MSG_CLOSE` | Server to client | Signals the end of a transfer. |
| `MSG_WORKER_JOIN` | Worker to server | Associates an enhanced worker socket with a session. |
| `MSG_WORKER_CLOS` | Server to worker | Signals that the worker channel has no more work. |
| `MSG_PULL_CHUNK` | Defined in Enhanced | Message identifier used for enhanced chunk requests. |

The shared utilities provide complete-send and complete-receive loops. This is important because TCP is a byte stream and a single socket call does not necessarily transfer a complete application message. The receiver first obtains the fixed-size header, validates its marker and payload length, and then reads the corresponding payload. This framing lets the application distinguish adjacent messages even when TCP splits one message across several reads or combines data from multiple sends. The implementation uses data chunks

of 32 KiB, balancing per-message overhead against the memory required for each transfer buffer.

## 5. Part I: Sequential and Concurrent Serving

### 5.1 Single-threaded mode

The single-threaded server accepts one client and handles its transfer before accepting the next client. This is straightforward to follow and provides a baseline. A slow client can delay later clients because the server does not handle them concurrently.

### 5.2 Multi-threaded mode

The multi-threaded server continues accepting clients and creates a worker thread for each accepted connection. A blocked socket operation for one client does not necessarily stop another worker from making progress. Threads improve concurrency but introduce scheduling and shared-state considerations.

### 5.3 Sample throughput graph

Figure 1 replaces the earlier connected-line chart with a point chart. It shows only the measurements that exist in `Results/sample_results.csv`. The sample contains one single-threaded measurement at one client and multi-threaded measurements at one, five, and ten clients. The single-threaded measurements for five and ten clients are missing, so this is not a complete comparison of the two modes.

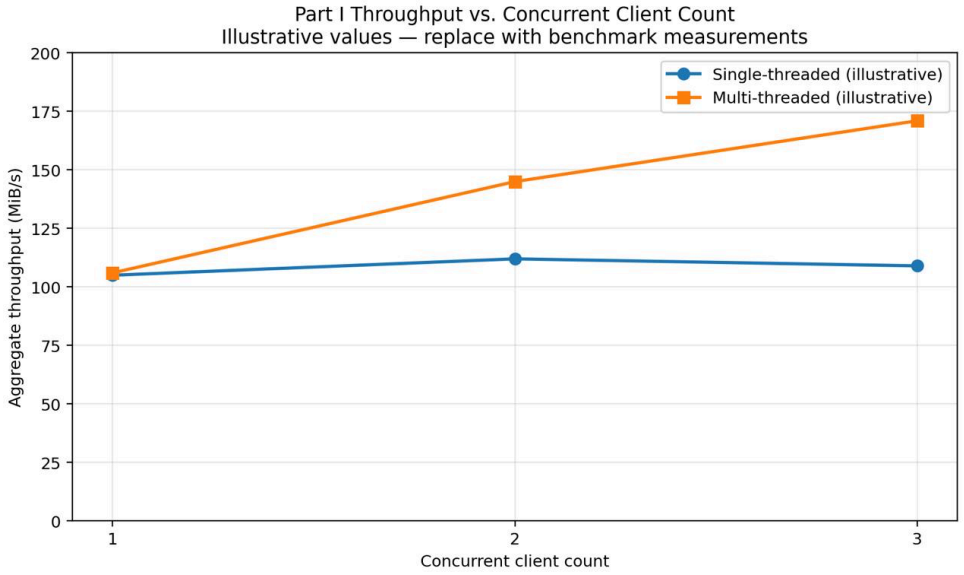



Figure 1: Part I throughput values in the supplied sample CSV. Missing single-threaded measurements are intentionally not estimated.

The horizontal axis is the number of concurrent clients and the vertical axis is the recorded throughput in the CSV. At one client, the sample records 160.59 for single-threaded mode and 111.63 for multi-threaded mode. The multi-threaded rows then report 183.15 at five clients and 198.85 at ten clients. These points suggest that aggregate throughput rises as more clients are served concurrently in this sample, but they do not establish that multi-threading is faster at every load: only the one-client point has both modes represented. The chart therefore deliberately leaves the missing single-threaded points blank rather than interpolating or inventing values.

For a fair comparison, both modes should be tested at the same client counts, with the same topic and dataset. Repeated trials should be used and the mean and standard deviation should be reported. The benchmark should also state whether throughput is decimal MB/s or MiB/s and calculate it consistently from useful bytes and elapsed time.

## 6. Part II Case 1: Fault Injection and Restart

The Case 1 server uses a configured probability to decide whether to interrupt a transfer at a chunk boundary. The source uses `rand_r()` for the fault draw in the acknowledgement-driven transfer loop. When a fault is triggered, the server logs it and closes the connection.

The client does not keep a resume checkpoint in this case. When a transfer fails, it discards incomplete output and requests the topic again from the beginning. This is simple, but it can resend data that the client already received.

| **Failure probability** | **Expected behaviour** |
|---|---|
| 0 | No injected failure; transfer should complete. |
| Low non-zero value | Some attempts may restart, depending on where a failure occurs. |
| Higher value | More restarts and more repeated data are expected. |
| 1 | Every eligible draw triggers a fault; completion is not expected. |

## 7. Part II Case 2: Session and Checkpoint Resume

Case 2 adds server-side session state and client checkpoint information. The session table is bounded and protected by a mutex. Session information includes a session ID, topic, file metadata, current file index, current byte offset, last-update time, and completion/activity flags. A cleanup thread removes completed or inactive sessions after the configured timeout.

The client tracks the session ID and the most recent file/offset checkpoint. After a connection failure, it reconnects and sends the checkpoint. The server validates the session and checkpoint before continuing. If the checkpoint cannot be accepted, the client must handle the error rather than assume that resume succeeded.

The main benefit is less repeated data. The trade-off is additional state and validation logic. A resume request must be checked against the session and the file/offset checkpoint; accepting an arbitrary offset could skip bytes or corrupt the reconstructed file. The session table is stored in memory, so it does not survive a server process restart. This design therefore supports recovery from connection interruption while the server process and its session state remain available; it is not persistent recovery across server restarts.

## 8. Case 2 Enhanced: Parallel Worker Connections

The enhanced variant extends the session-based design with multiple TCP worker connections. A primary channel establishes the session, and worker channels join it to receive work. The protocol and server structures track chunk tasks and their progress.

The client prepares destination files and uses `ftruncate()` to set their expected size. It uses `pwrite()` to write data at an explicit offset. This allows different workers to write different parts of a file without sharing one mutable file cursor. Correctness depends on the assigned ranges not overlapping incorrectly and on failed tasks being retried or re-queued.

Parallel workers can improve utilisation when one stream is limited by latency or per-request turnaround. They also add thread, socket, and coordination overhead, and may contend for disk access or server CPU. Chunk scheduling must ensure that every byte range is assigned and completed, while the client must only consider a file complete after all required ranges have been received. The sample data alone does not prove that the enhanced version is faster for every workload; it should be evaluated across multiple file sizes and worker counts.

## 9. Input Validation and Concurrency

The implementation includes name validation, message-type checks, payload-size checks, and error responses for invalid requests. The utility routines handle partial socket reads/writes and interrupted calls. The session table and enhanced task structures require synchronization because multiple worker threads may access shared state.

All four variants built successfully in the report-generation environment. The enhanced and stateful variants emitted some string/path truncation warnings. These warnings did not stop the build, but the code should be reviewed to ensure that composed paths and copied strings are checked for length before use.

## 10. Build and Run Guide

Build each variant from the project root:

```
make -C Part1
make -C Part2_Case1
make -C Part2_Case2
make -C Part2_Case2_Enhanced
```

Example local run:

```
# Terminal 1: start the Part I server
./Part1/server 18001 /path/to/topics multi

# Terminal 2: request a topic
./Part1/client 127.0.0.1 18001 cane ./received_files

# Example Part II server with failure probability 0.05
./Part2_Case2/server 18003 /path/to/topics 0.05
```

Use a free port for each server process. The requested topic must exist as a subdirectory beneath the supplied root. Exact command-line details should be checked against the README and each variant's source before the final submission.

The archive's benchmark runner is in `Results/benchmark.py`; plotting code is in `Results/plot_results` The runner expects an external test dataset that is not included in the ZIP. It also contains fallback logic that can substitute a default byte count if parsing fails. A reliable benchmark should fail clearly when parsing fails instead of silently inserting a value.

## 11. Performance Metrics and Sample Results

The report uses these metric definitions:

- **Useful bytes:** data successfully delivered and retained by the client.
- **Redundant bytes:** bytes retransmitted beyond the useful dataset.
- **Retransmission overhead:** redundant bytes divided by useful bytes, multiplied by 100%.
- **Effective throughput:** useful bytes divided by elapsed transfer time.
- **Completion time:** elapsed time from transfer start until completion.

The tables and graphs in this section use values copied from the supplied `Results/sample_results.csv`. They are sample results, not fresh benchmark runs from this report. The throughput values are reproduced as recorded in the CSV; the unit convention should be confirmed against the benchmark implementation before final submission.

### 11.1 Part II throughput

The throughput graph compares the restart-based Case 1 client with the checkpoint-resume and enhanced variants across the failure probabilities represented in the CSV. Case 1 throughput falls sharply as failures become more frequent: repeated transfers increase completion time while the useful file size remains fixed. Case 2 and Enhanced also show lower throughput at higher failure probabilities, even though their recorded redundant-byte counts remain zero. This is plausible because reconnecting, validating session state, coordinating workers, and waiting for a transfer to finish all consume time that is not

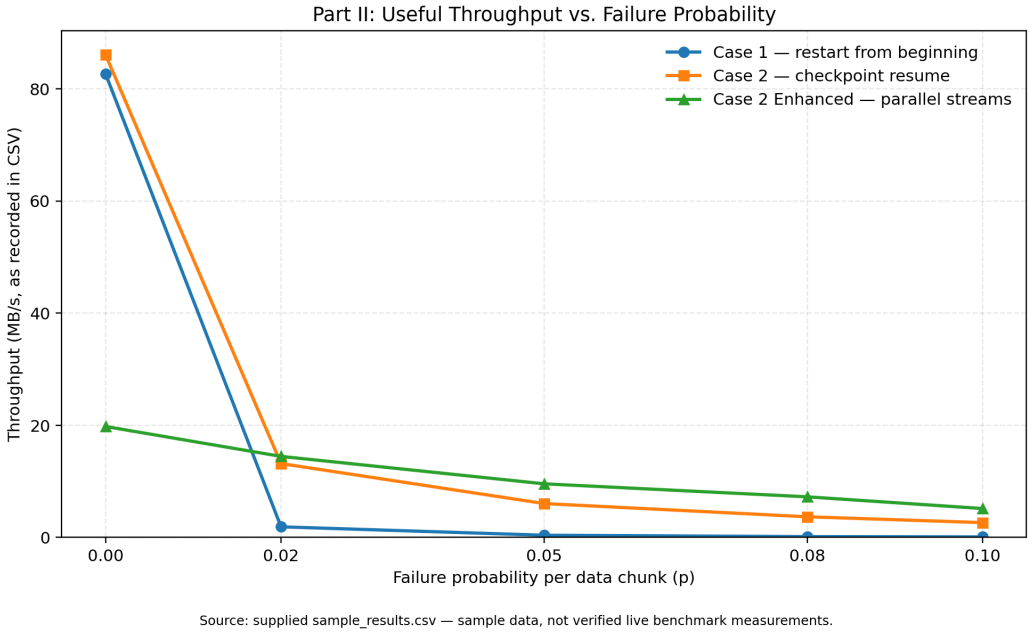



Figure 2: Part II throughput values from the supplied sample CSV; the plotted values have not been independently reproduced.

counted as redundant payload. The plot should be read as a summary of the supplied sample rows, not as a verified benchmark result.

### 11.2 Part II retransmission overhead

The overhead graph reports redundant bytes as a percentage of useful bytes, not as a percentage of elapsed time. The inset is included because the Case 2 and Enhanced rows are recorded as exactly 0%, which otherwise would be hard to distinguish along the same vertical scale as the large Case 1 values. Case 1 rises to 100% at failure probability 0.02, 400% at 0.05, 1075.05% at 0.08, and 1500% at 0.10. In practical terms, an overhead of 400% means that the recorded redundant payload is four times the useful payload.

The zero-overhead values are consistent with successful resume from the last validated checkpoint, but the CSV alone cannot prove that the counters are correct. The benchmark code contains fallback behaviour that may insert a default byte count when parsing fails. Therefore, these zero values should be treated as unverified until client/server logs and byte counters are checked against the actual output files. A zero retransmission overhead also does not imply zero recovery cost: connection setup and resume validation can still reduce effective throughput.

## 12. Verification and Limitations

### 12.1 Build and smoke test

The four variants compiled successfully. A small local fixture was also used for a functional smoke test: topic `cane` contained `a.txt` (19 bytes) and `b.bin` (76,800 bytes), for a total

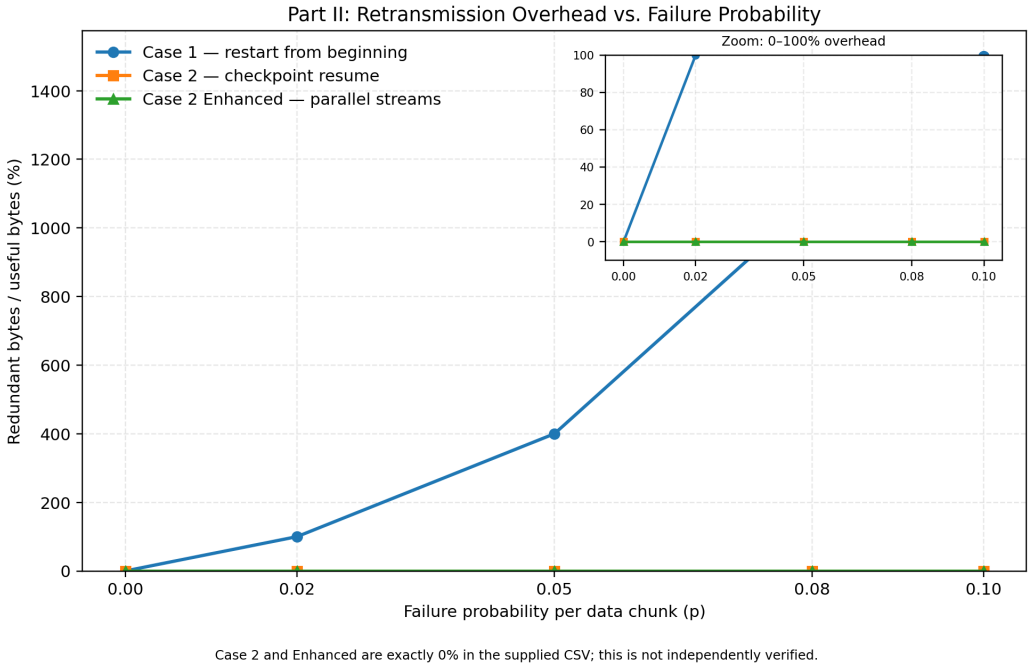



Figure 3: Retransmission overhead values from the supplied sample CSV, with a zoomed view to make low values visible.

of 76,819 bytes. With failure probability zero, each variant completed and the output files matched the input file sizes. This was a functional check, not a performance benchmark and not a test of non-zero fault recovery.

| **Variant** | **Observed outcome** |
|---|---|
| Part I single-threaded | Completed; both files received. |
| Part I multi-threaded | Completed; both files received. |
| Part II Case 1, $p = 0$ | Completed; no injected fault. |
| Part II Case 2, $p = 0$ | Completed; no resume event needed. |
| Case 2 Enhanced, $p = 0$ | Completed with two workers requested. |

### 12.2 Known limitations

- The benchmark dataset expected by the runner is not included in the archive.
- The sample CSV should not be treated as a fresh measurement without raw logs and a repeatable run.
- The benchmark's fallback byte count can hide a parsing failure and should be removed.
- The Part I sample lacks single-threaded measurements at five and ten clients.
- Some path/string truncation warnings remain in the stateful and enhanced code.
- File sizes were checked in the smoke test; full content hashes and high-concurrency failure tests remain to be run.

### 12.3 Recommended benchmark procedure

1. Provide the assignment dataset and record the topic, file count, and total bytes.
2. Build all four variants from a clean state and save compiler output.
3. Run both Part I modes at identical client counts and repeat each configuration.
4. For Part II, use the same topic and a documented failure-probability sweep for all variants.
5. Save client logs, server logs, exit codes, and raw CSV rows from each run.
6. Check received files against source sizes and hashes.
7. Recompute throughput and overhead from validated counters; do not replace failed parsing with a default value.
8. Regenerate every graph directly from the validated CSV.

## 13. Design Choices

| **Decision** | **Alternative** | **Reason and trade-off** |
|---|---|---|
| Single vs. multi-threaded | One event loop | Threads follow the assignment's client concurrency model and simplify blocking I/O; they add thread overhead. |
| Case 1 restart | Preserve partial files | Restarting is simple but can retransmit previously received data. |
| Case 2 checkpoint | No session state | Resume avoids repeating completed data but requires state validation and synchronization. |
| Chunked transfer | Send whole files as one payload | Chunks make progress tracking and fault injection more precise, but increase message count. |
| Parallel worker connections | One connection per topic | Multiple workers can overlap work but require scheduling and safe offset writes. |
| Explicit-offset writes | Shared file pointer | `pwrite()` lets workers write at chosen offsets without racing on a shared cursor. |
| Session cleanup | Keep all sessions indefinitely | A bounded table and timeout reaper reclaim stale state; in-memory state is lost on server restart. |

## 14. Conclusion

The submitted archive contains four implementations of a TCP topic-based file distribution service. Part I provides sequential and concurrent serving. Case 1 demonstrates restart after a connection failure. Case 2 adds session state and checkpoint-based resume. Case 2 Enhanced adds parallel worker connections and explicit-offset writes.

The builds and zero-fault smoke tests completed successfully. The archive also contains sample performance data, but the missing dataset, fallback accounting, and incomplete Part I client-count comparison mean the sample figures should be labelled clearly and revalidated. The next step is to run a controlled benchmark on the assignment dataset and regenerate the results from validated measurements.

## A. Protocol Constants and Bounds

| Constant | Value | Use |
|---|---|---|
| `PROTOCOL_MAGIC` | `0x49543330` | Header validation. |
| `MAX_TOPIC_NAME` | 128 bytes | Topic field capacity. |
| `MAX_FILENAME_LEN` | 256 bytes | Filename field capacity. |
| `CHUNK_SIZE` | 32 KiB | Data chunk size. |
| `MAX_REASONABLE_PAYLOAD` | 16 MiB | Utility-level payload bound. |
| `MAX_FILES_PER_TOPIC` | 8,192 | Per-topic file bound in server code. |
| `MAX_SESSIONS` | 32 | Case 2 session-table capacity. |
| `SESSION_TIMEOUT_SEC` | 300 seconds | Session expiry threshold. |
| Default parallel connections | 4 | Enhanced client default. |

## B. Archive Inventory

The ZIP contains the four implementation folders, each with C source, protocol definitions, utility code, and a Makefile. The `Results/` folder contains the benchmark runner, plotting script, data helper, sample CSV, and existing SVG charts. The README and Markdown report template provide setup notes but still contain placeholder group identifiers.